

TEA: Time-Proportional Event Analysis

50th International Symposium on Computer Architecture (ISCA)

Björn Gottschall (NTNU), Lieven Eeckhout (Ghent) and Magnus Jahre (NTNU)



NTNU

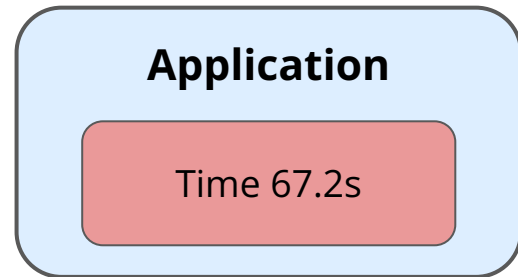
Norwegian University of
Science and Technology



GHENT
UNIVERSITY

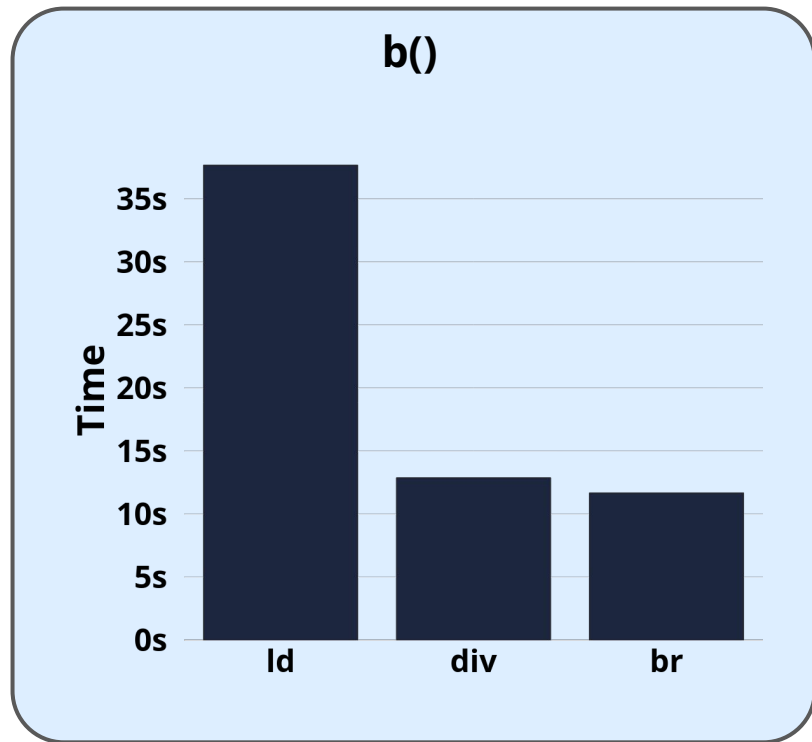
The Goal of Performance Profiling

- How much time does it take to execute my application?
- Where does the application spend most of its execution time?
 - Which are the most performance critical instructions?
- Why are these instructions performance critical?
 - What microarchitectural events are the cause of performance loss?



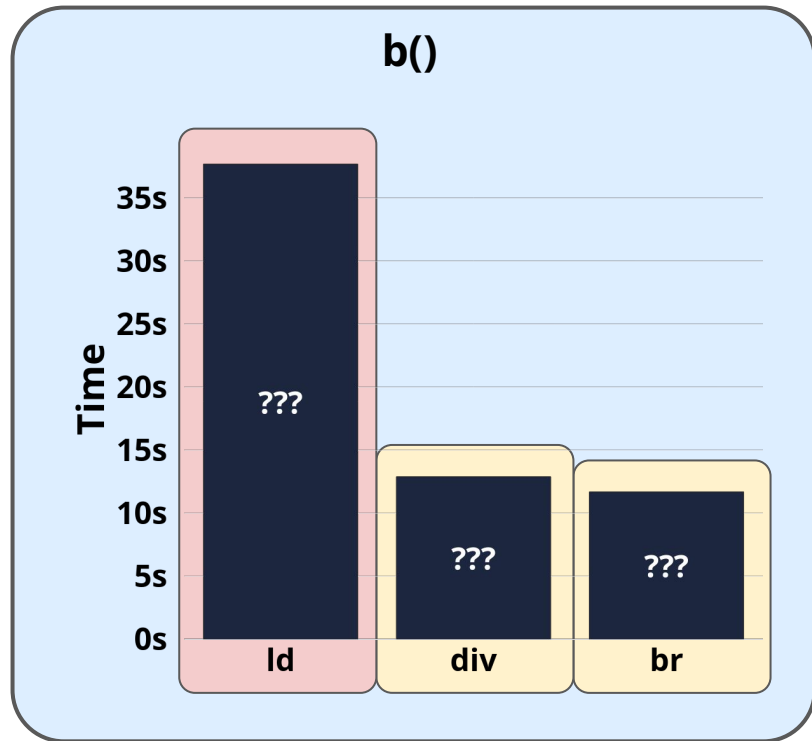
The Goal of Performance Profiling

- How much time does it take to execute my application?
- Where does the application spend most of its execution time?
 - Which are the most performance critical instructions?
- Why are these instructions performance critical?
 - What microarchitectural events are the cause of performance loss?



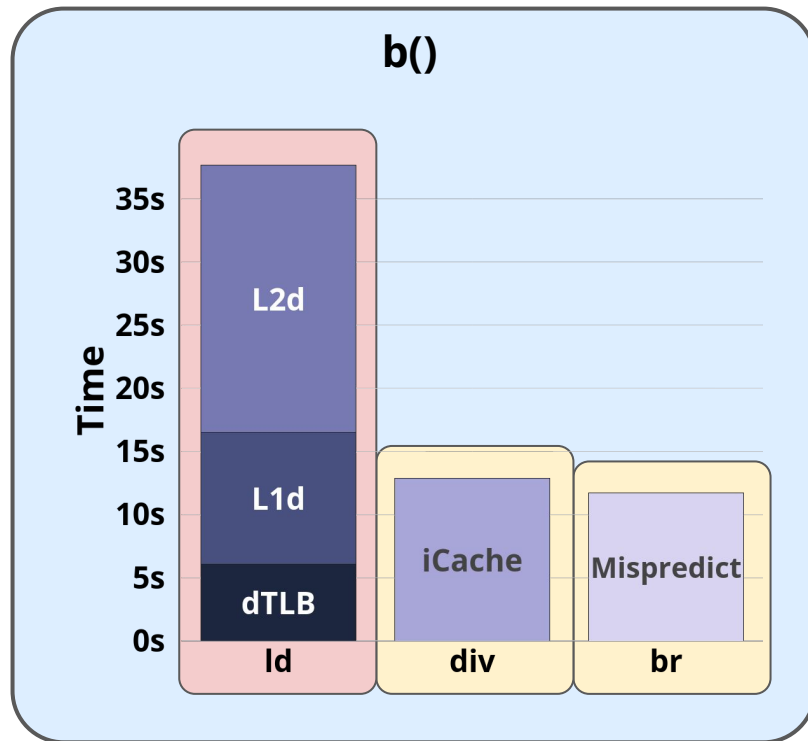
The Goal of Performance Profiling

- How much time does it take to execute my application?
- Where does the application spend most of its execution time?
 - Which are the most performance critical instructions?
- Why are these instructions performance critical?
 - What microarchitectural events are the cause of performance loss?



The Goal of Performance Profiling

- How much time does it take to execute my application?
- Where does the application spend most of its execution time?
 - Which are the most performance critical instructions?
- Why are these instructions performance critical?
 - What microarchitectural events are the cause of performance loss?



The Goal of Performance Profiling

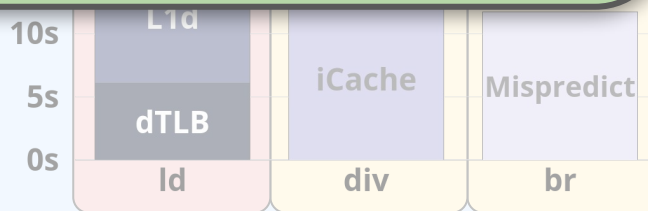
- How much time spends my application

b()

**TEA: Time-Proportional Event Analysis
answers all of these questions!**

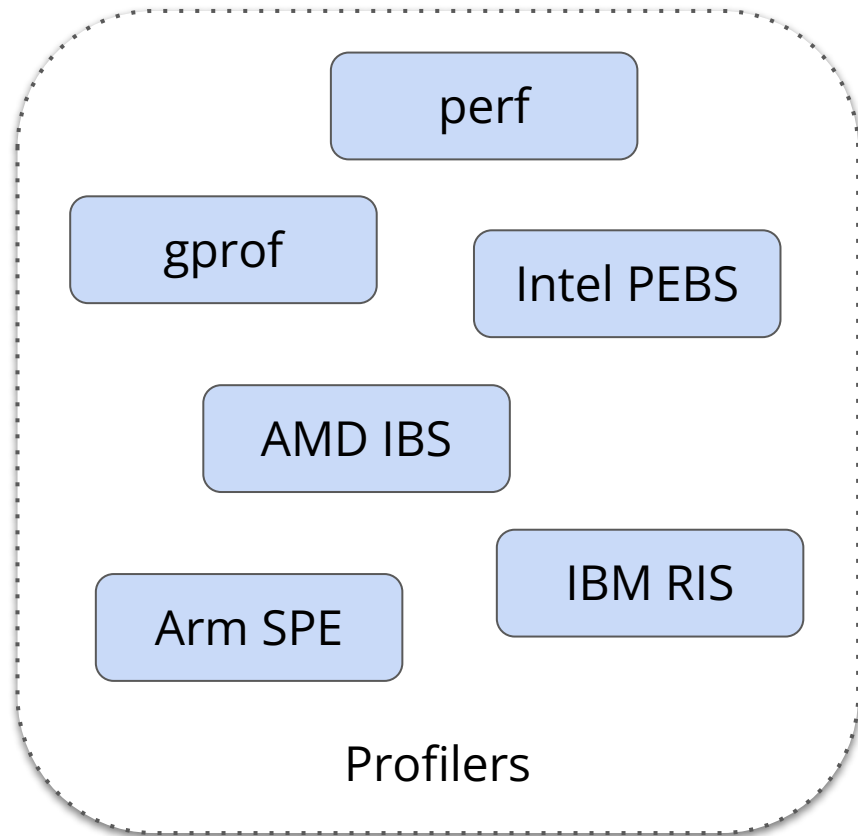
Critical:

- What microarchitectural events are the cause of performance loss?



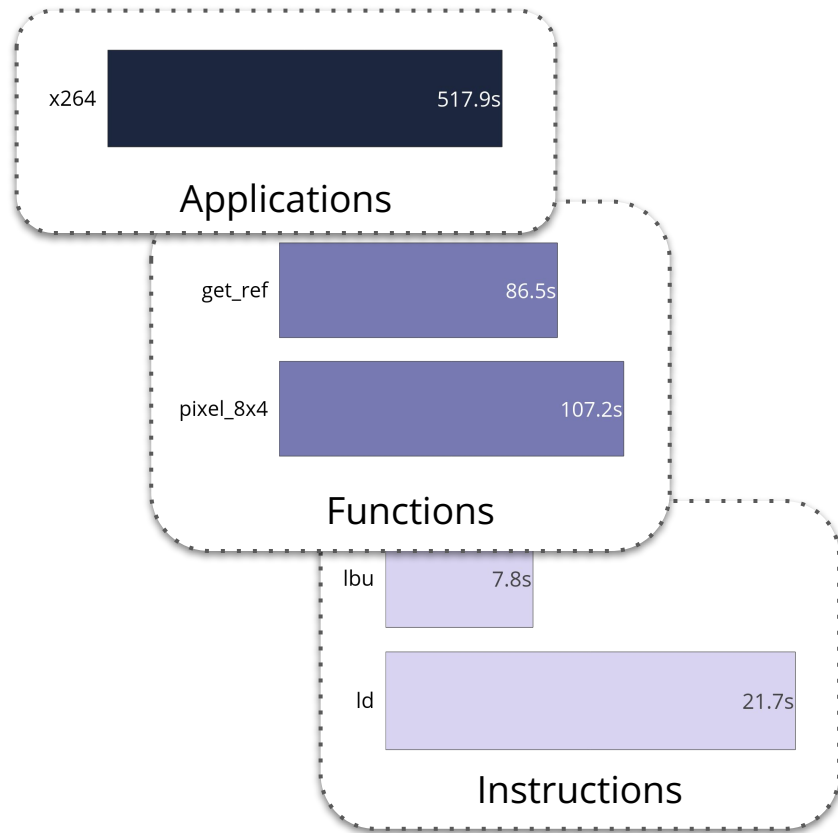
TEA: Unified Performance Analysis

- Performance profilers have been around for a long time
- Many break down time across
 - Applications
 - Functions
 - Instructions
- Some record performance events
 - Counting
 - Attribution to instructions



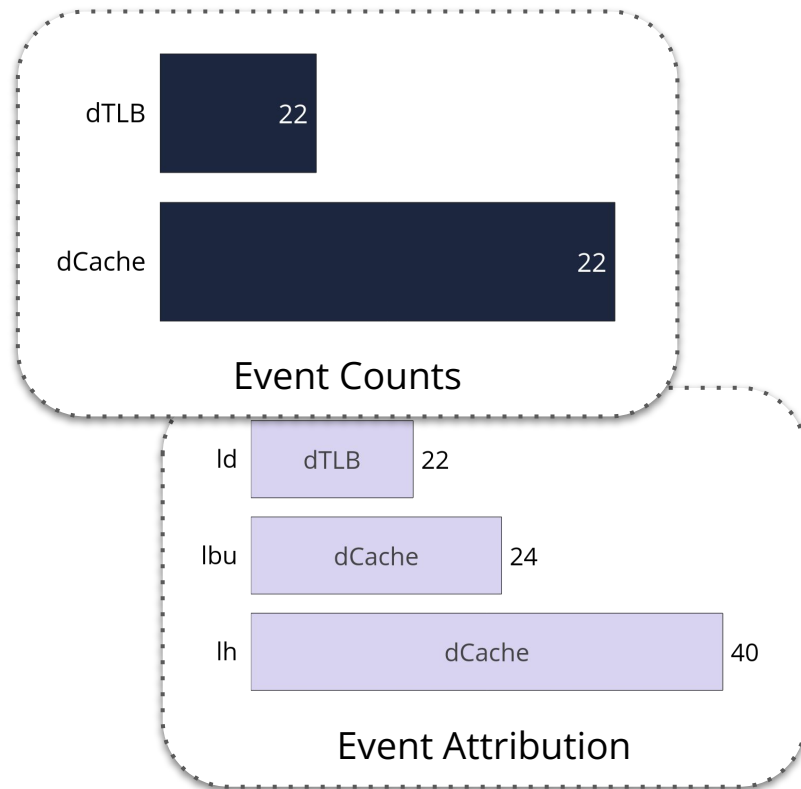
TEA: Unified Performance Analysis

- Performance profilers have been around for a long time
- Many break down time across
 - Applications
 - Functions
 - Instructions
- Some record performance events
 - Counting
 - Attribution to instructions



TEA: Unified Performance Analysis

- Performance profilers have been around for a long time
- Many break down time across
 - Applications
 - Functions
 - Instructions
- Some record performance events
 - Counting
 - Attribution to instructions



TEA: Unified Performance Analysis

- Performance profilers have been around for decades

TEA captures all of this information accurately in a single run with negligible overhead

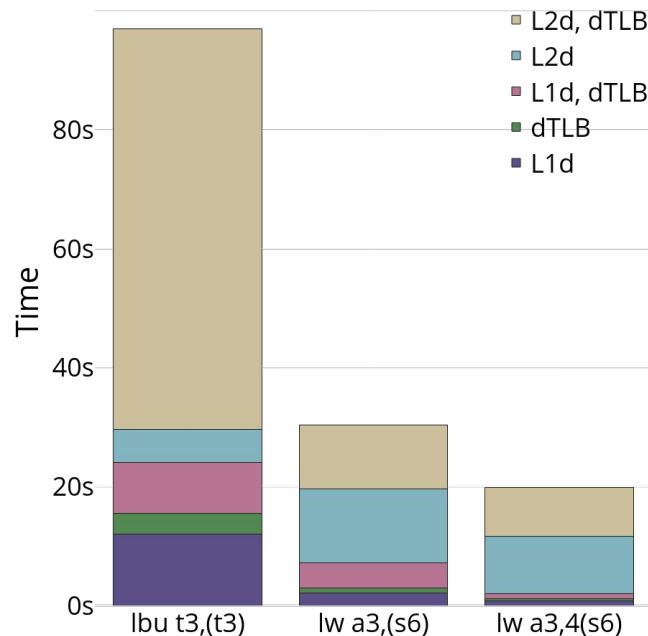
- Some record performance events
 - Counting
 - Attribution to instructions



Event Attribution

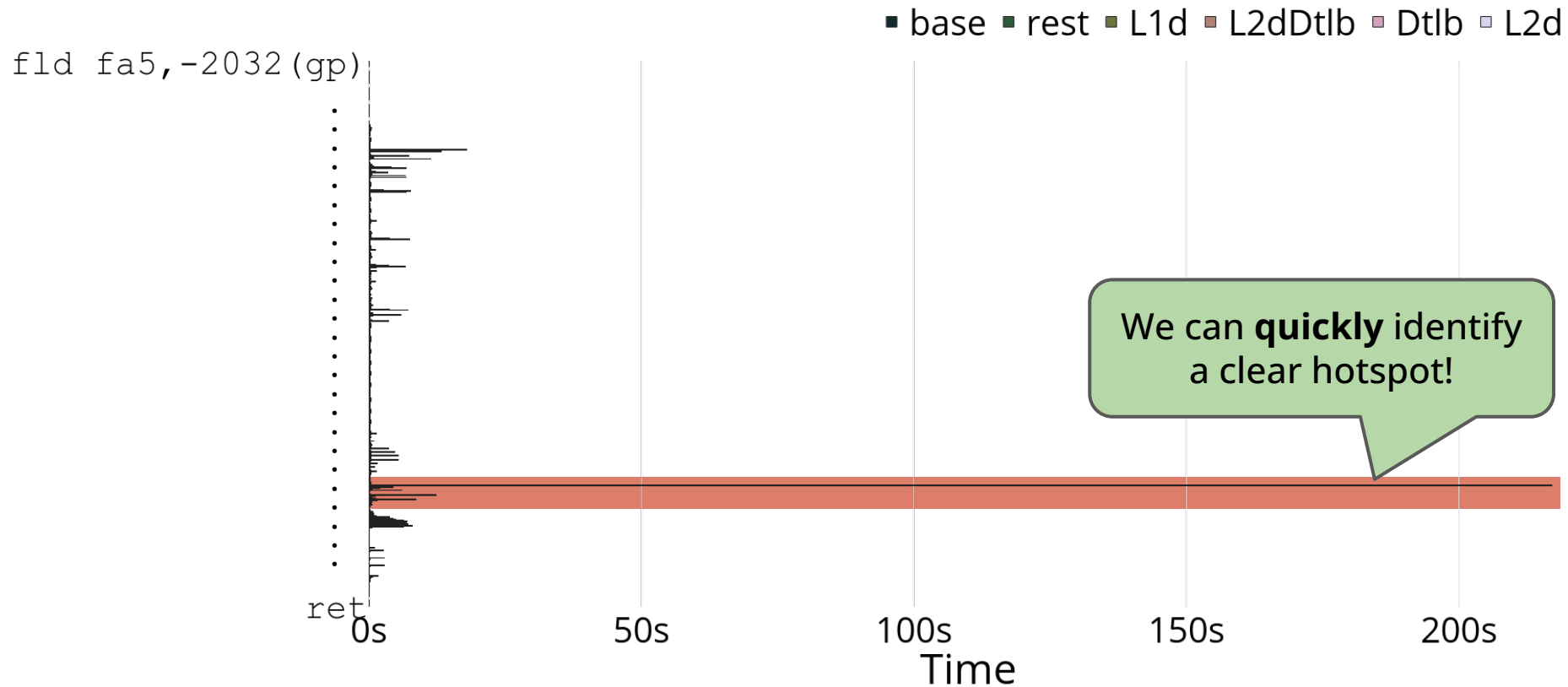
TEA – Unified Performance Analysis

- **Time-Proportional Event Analysis** is the **first** unified performance analyzer
- **Per-Instruction Cycle Stacks (PICS)**
- It is **accurate** with 2.1% average error
- **Negligible** storage (249 bits), power (0.1%) and runtime (1.1%) **overhead**



Per-Instruction Cycle Stacks

TEA in Action – Optimizing LBM



TEA in Action – Optimizing LBM

■ base ■ rest ■ L1d ■ L2dDtlb ■ Dtlb ■ L2d

lw t4,152(a0)

:

TEA: This load is **performance-critical**, mostly due to **non-hidden L2 misses**

:

TEA maps the load to the first access to the source grid in LBM

fld ft1,88(a0)

0s

50s

100s

Time

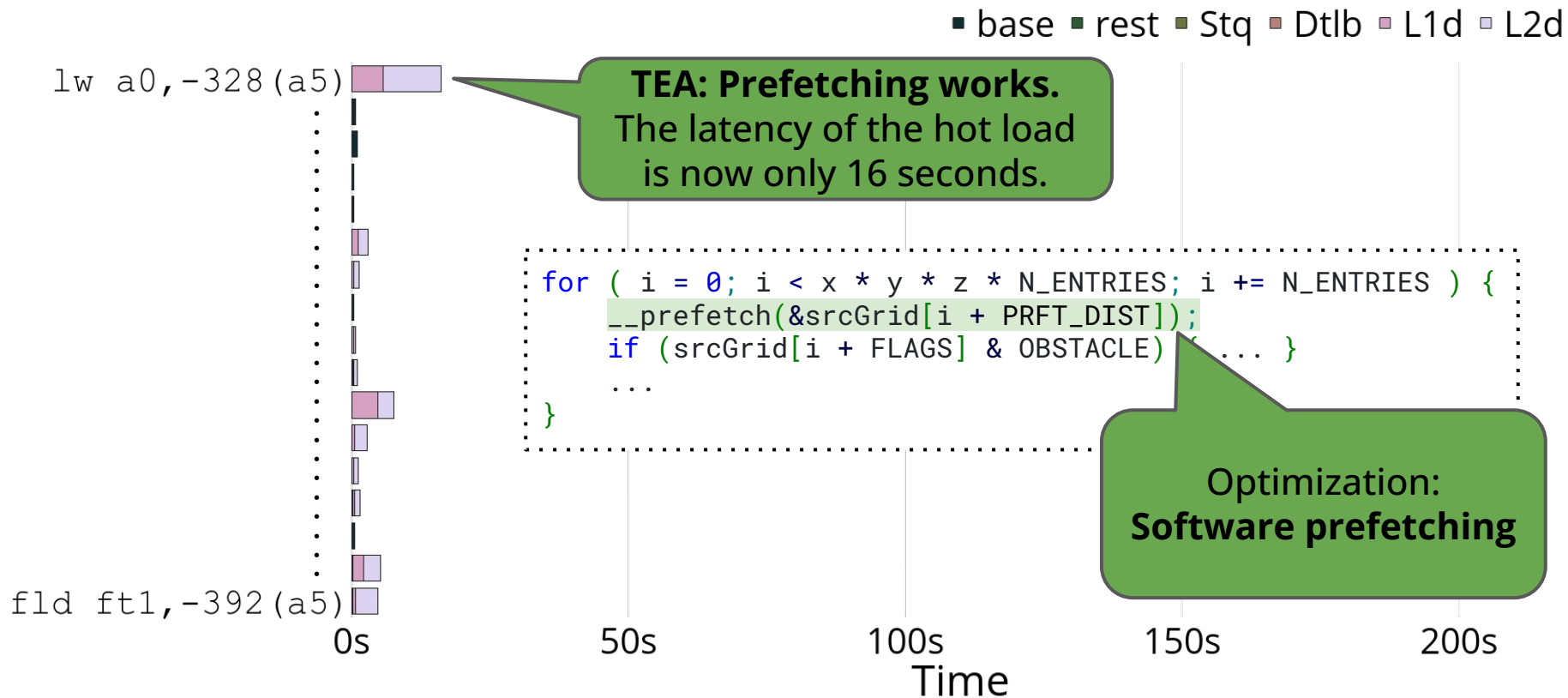
150s

200s

```
for ( i = 0; i < x * y * z * N_ENTRIES; i += N_ENTRIES ) {  
    __prefetch(&srcGrid[i + PRFT_DIST]);  
    if (srcGrid[i + FLAGS] & OBSTACLE) { ... }  
}
```

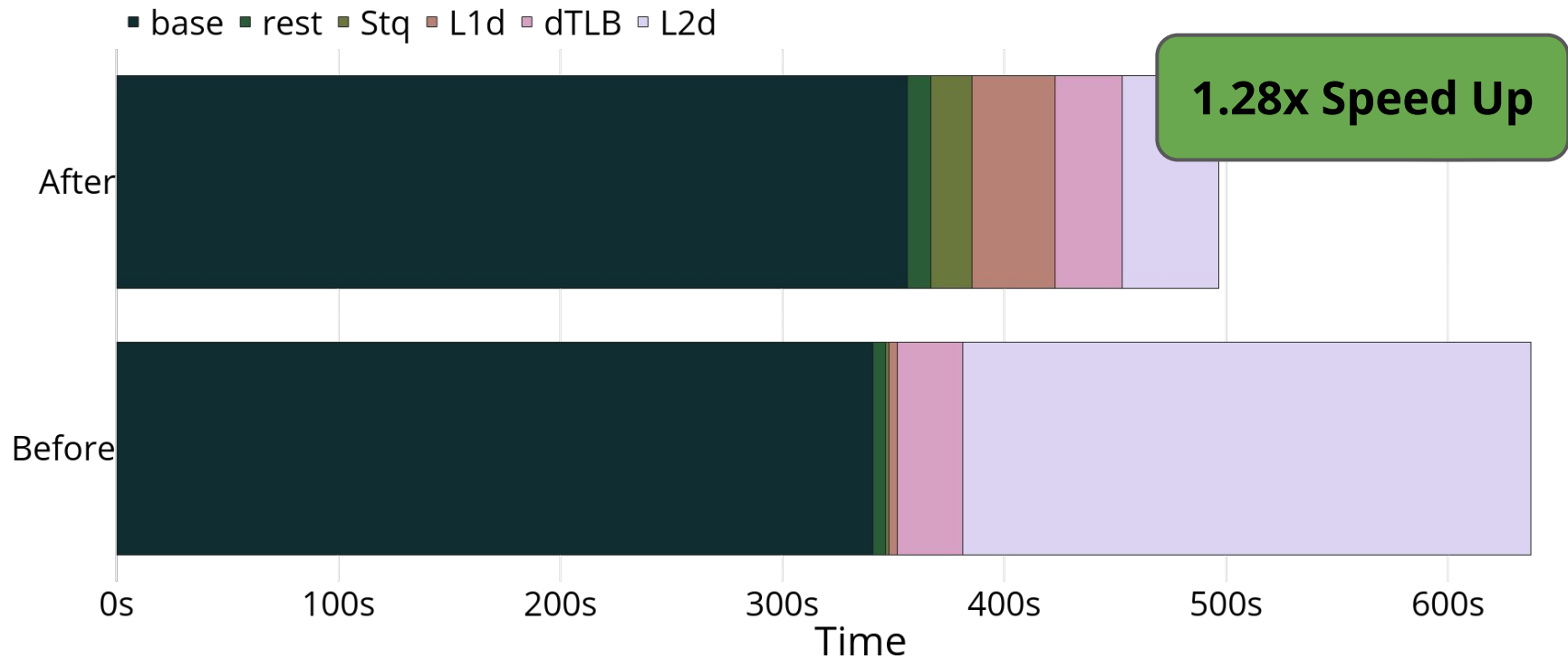
Optimization:
Software prefetching

TEA in Action – Optimizing LBM

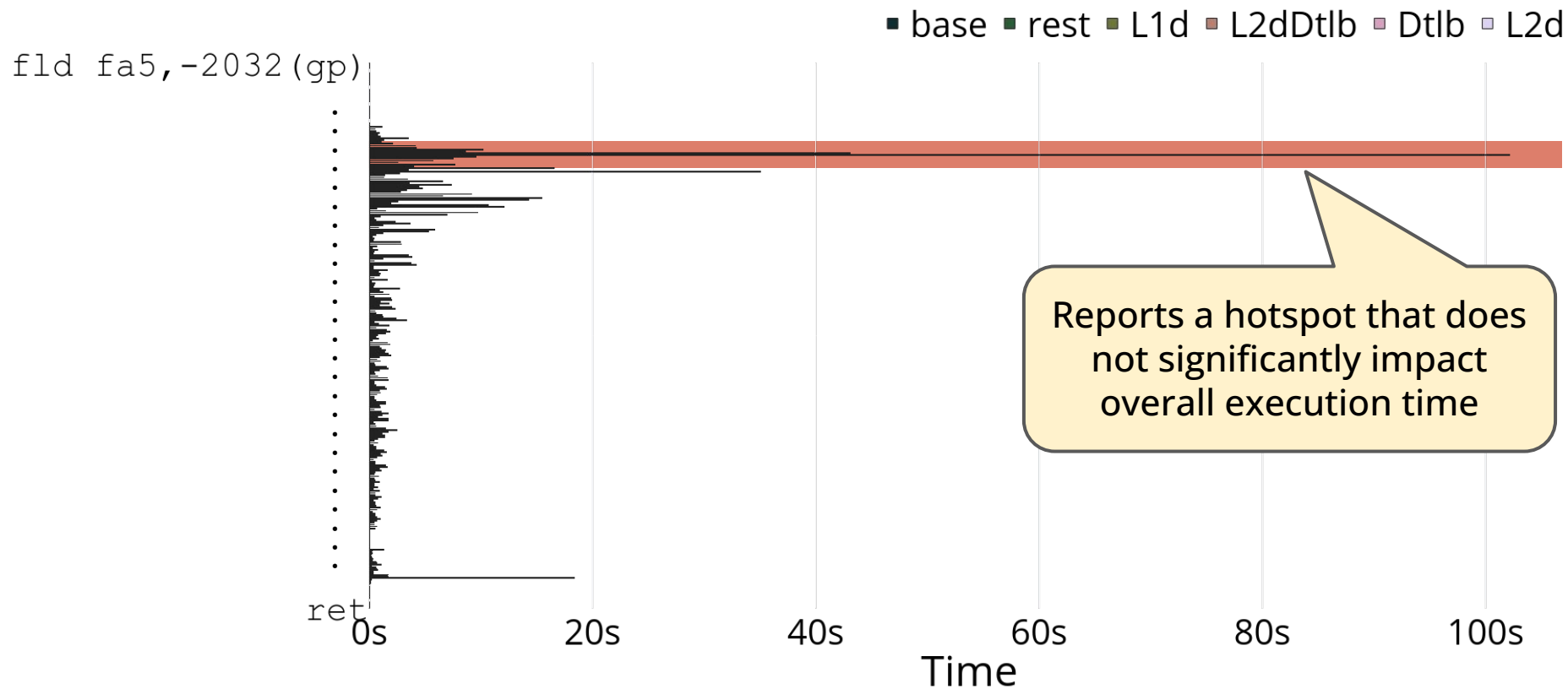


TEA in Action – Optimizing LBM

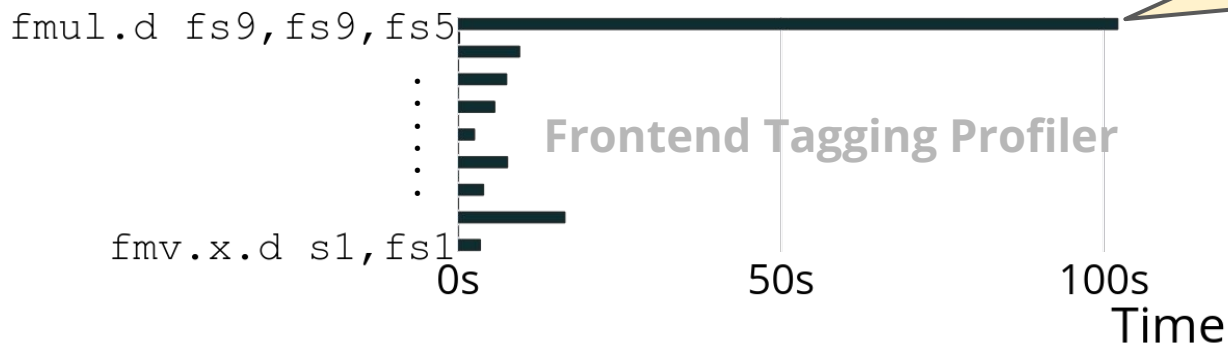
Application-level Cycle Stacks



Current State-of-the-Art Analyzers

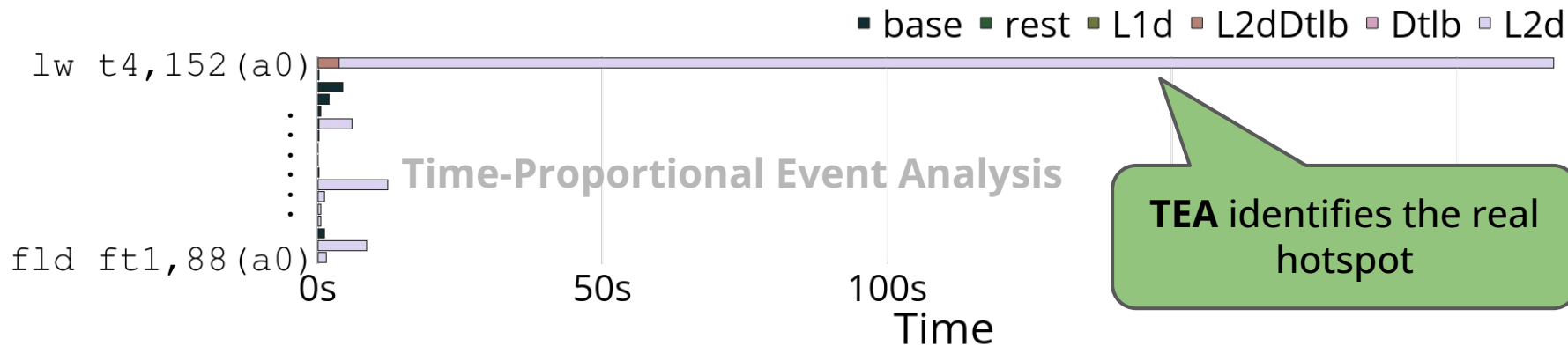


Current State-of-the-Art Analyzers



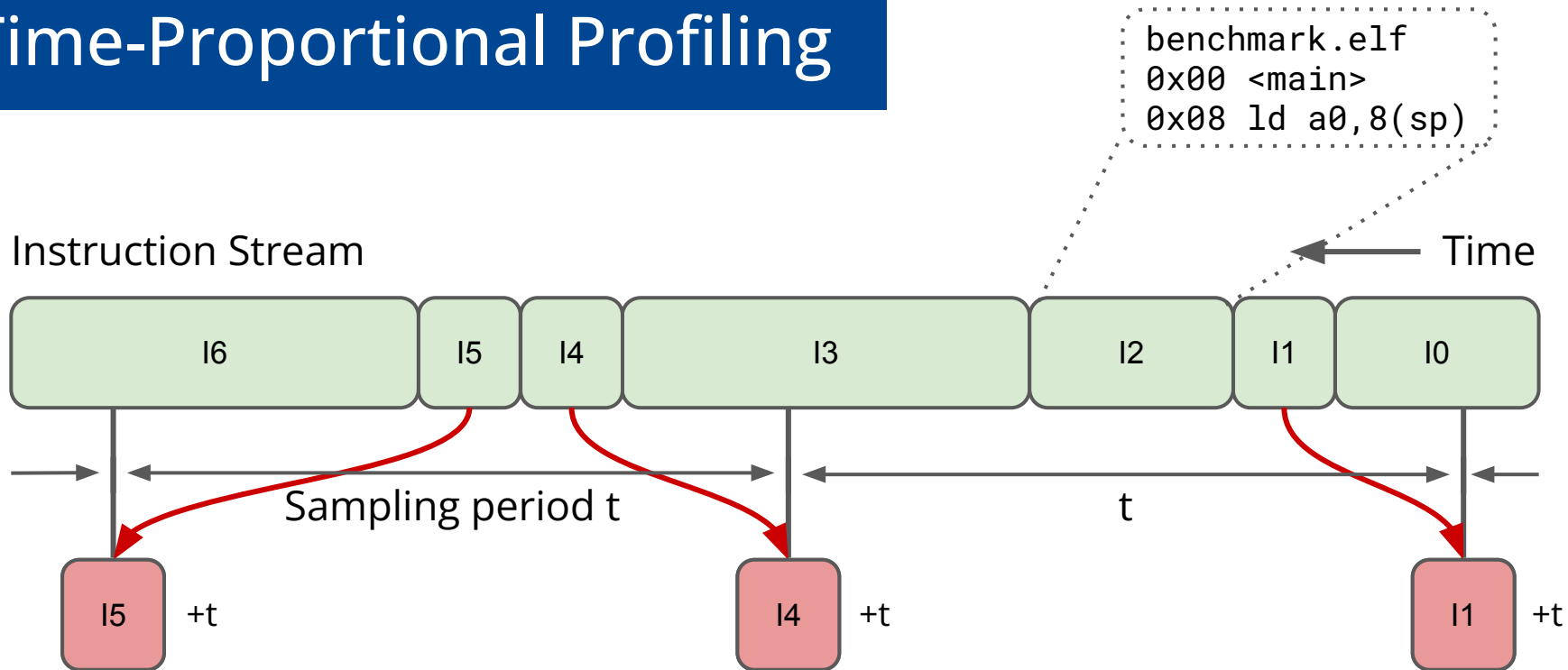
Focuses on **instructions** without significant impact on execution time

Why?
State-of-the-art analyzers are **not time-proportional**



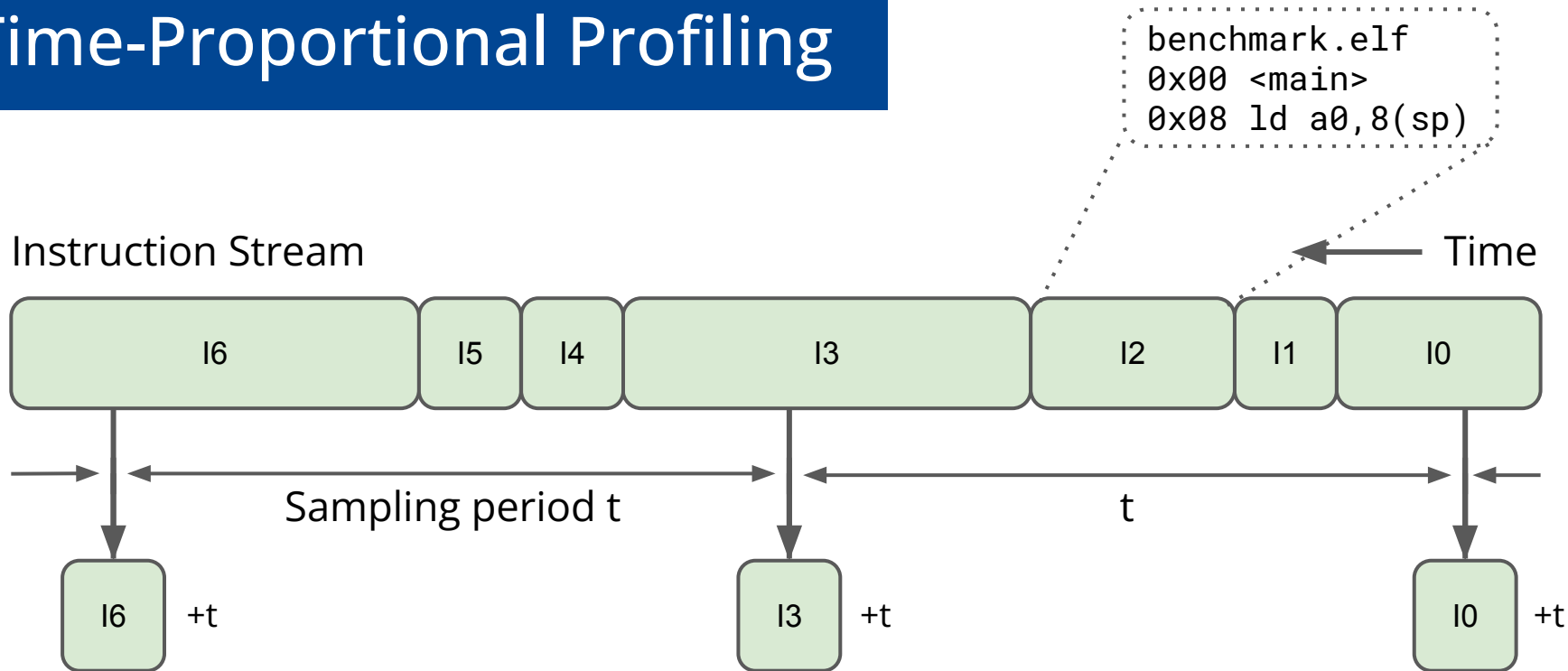
TEA identifies the real hotspot

Time-Proportional Profiling



→ Runtime is distributed over collected samples to create a **performance profile**

Time-Proportional Profiling

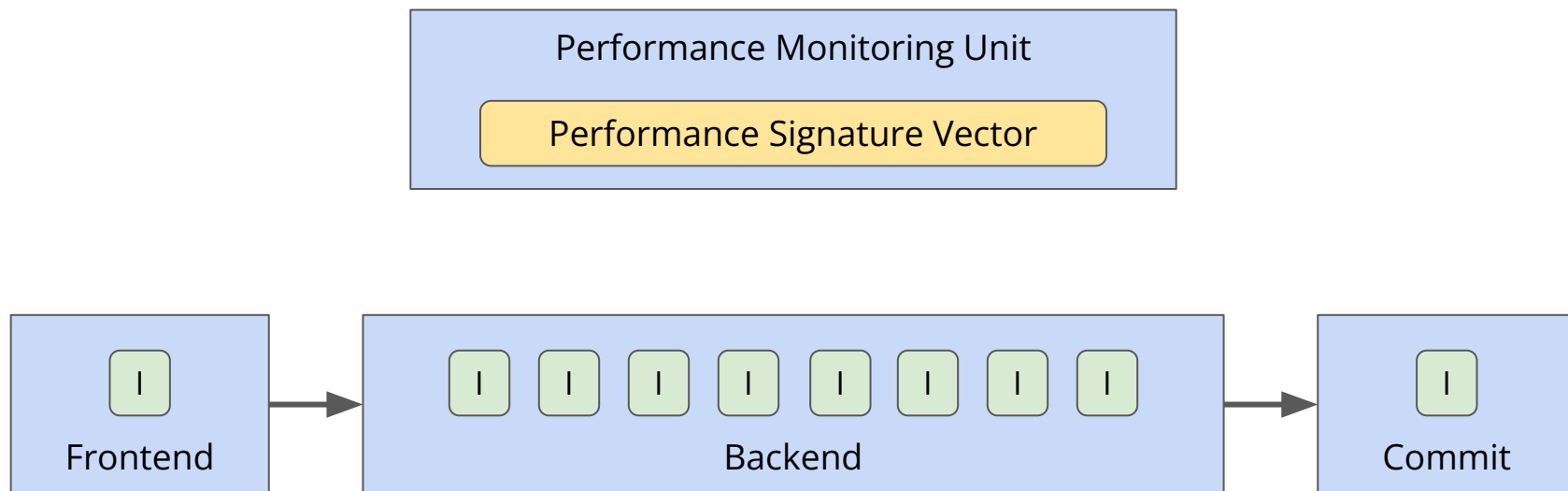


- Runtime is distributed over collected samples to create a **performance profile**
- **Time-Proportionality Principle**: The probability of sampling an instruction must be proportional to its relative contribution to overall execution time¹

¹ TIP: Time-Proportional Instruction Profiling, MICRO 2021

State of the Art – Performance Profiling

Frontend Instruction Tagging¹



Instruction Legend



Normal



Stalled

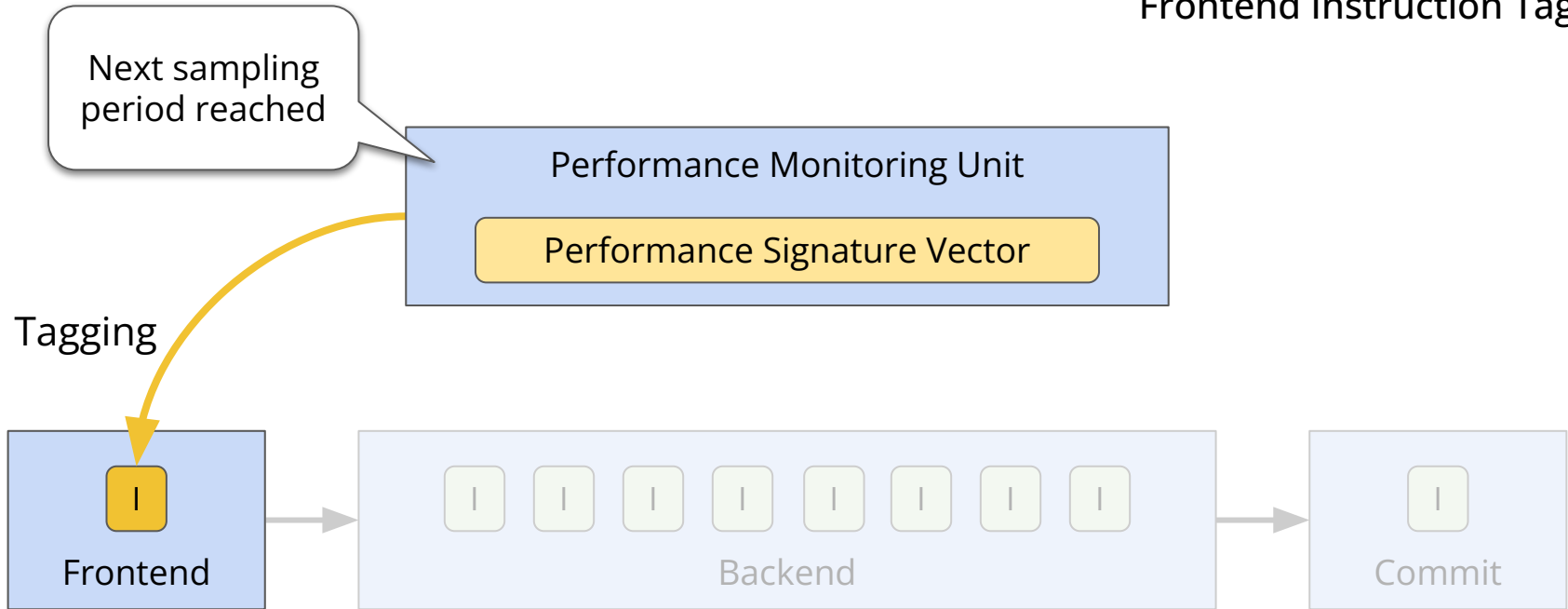


Tagged

¹ e.g. AMD IBS, Arm SPE or IBM RIS

State of the Art – Performance Profiling

Frontend Instruction Tagging¹



Instruction Legend



Normal



Stalled

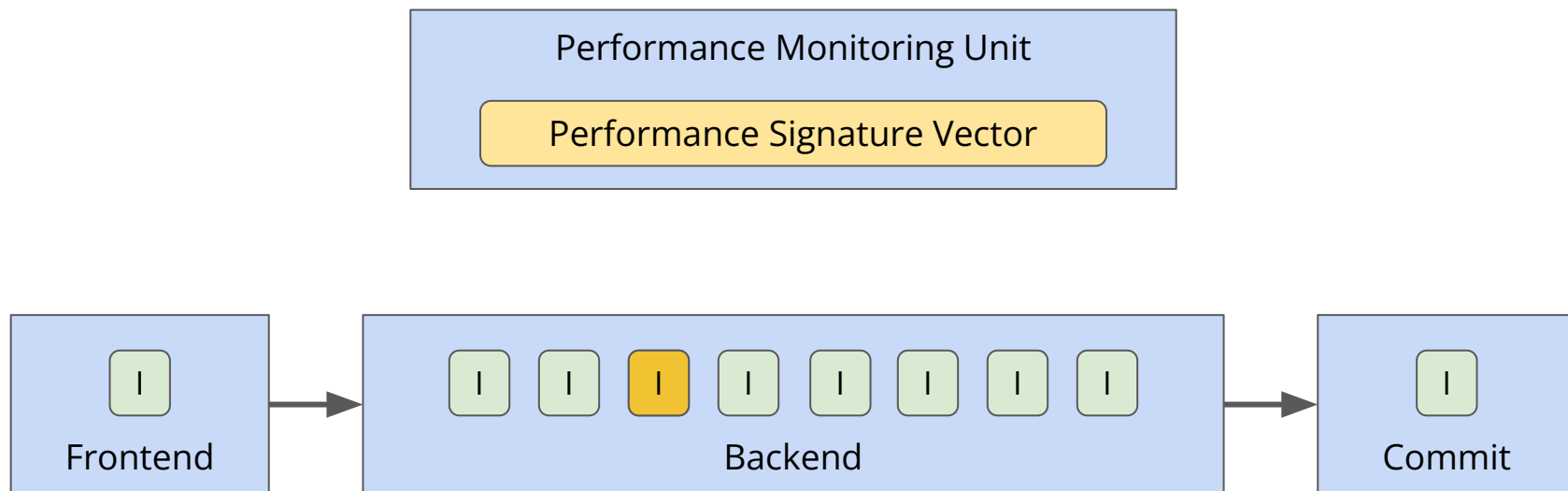


Tagged

¹ e.g. AMD IBS, Arm SPE or IBM RIS

State of the Art – Performance Profiling

Frontend Instruction Tagging¹



Instruction Legend



Normal



Stalled

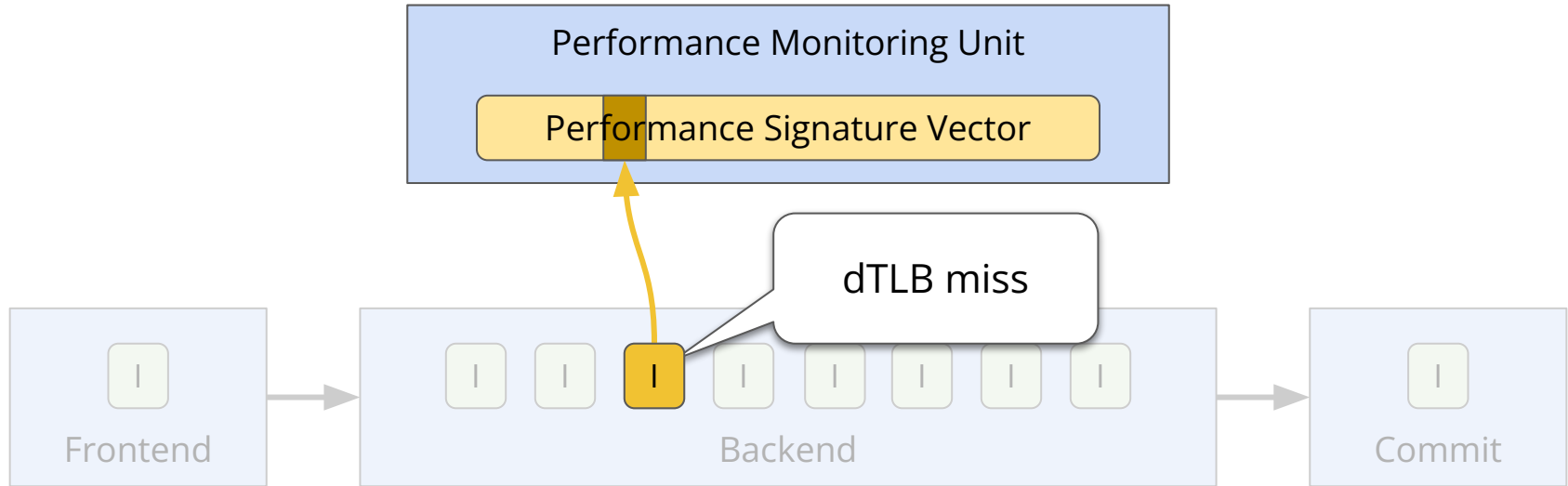


Tagged

¹ e.g. AMD IBS, Arm SPE or IBM RIS

State of the Art – Performance Profiling

Frontend Instruction Tagging¹



Instruction Legend



Normal



Stalled

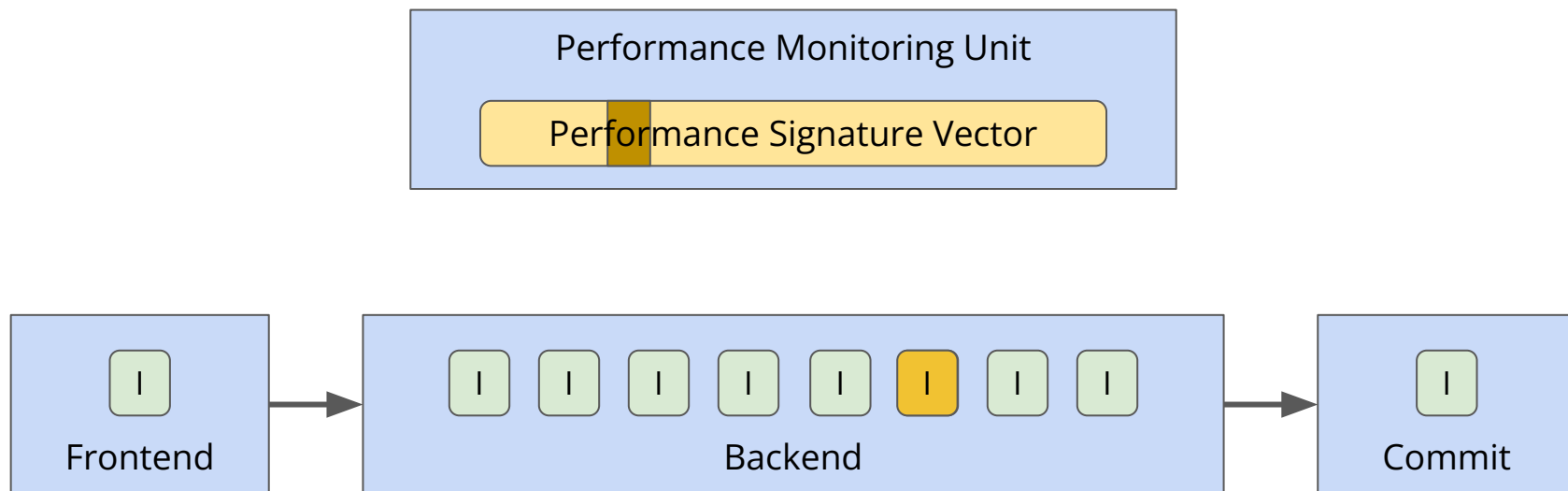


Tagged

¹ e.g. AMD IBS, Arm SPE or IBM RIS

State of the Art – Performance Profiling

Frontend Instruction Tagging¹



Instruction Legend



Normal



Stalled

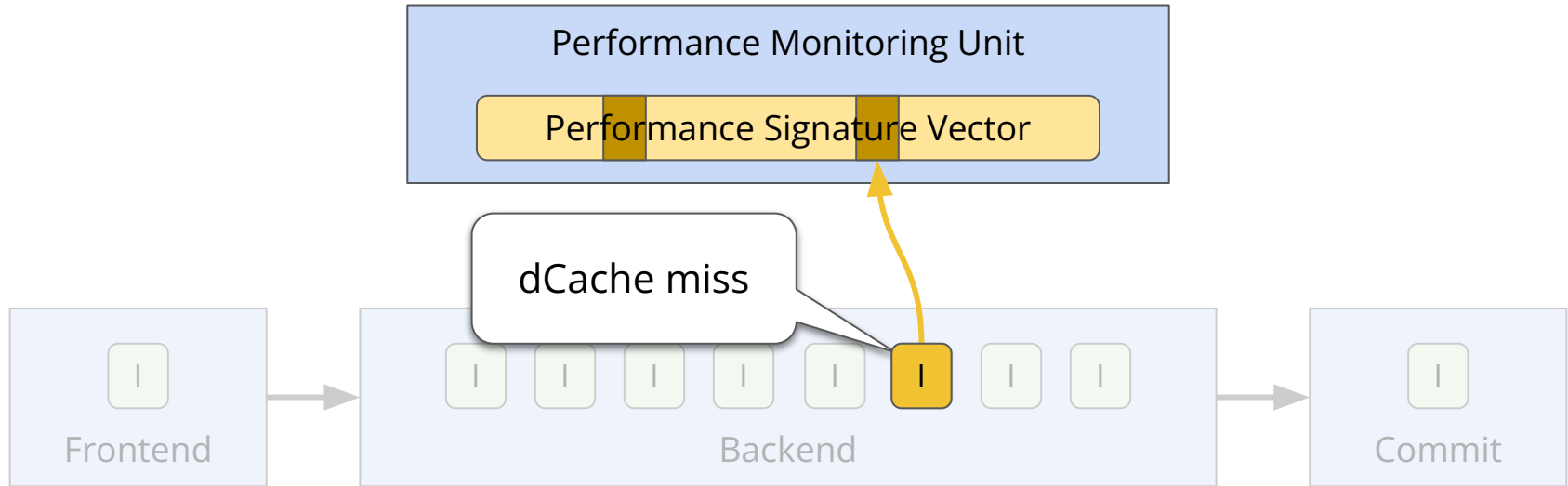


Tagged

¹ e.g. AMD IBS, Arm SPE or IBM RIS

State of the Art – Performance Profiling

Frontend Instruction Tagging¹



Instruction Legend



Normal



Stalled

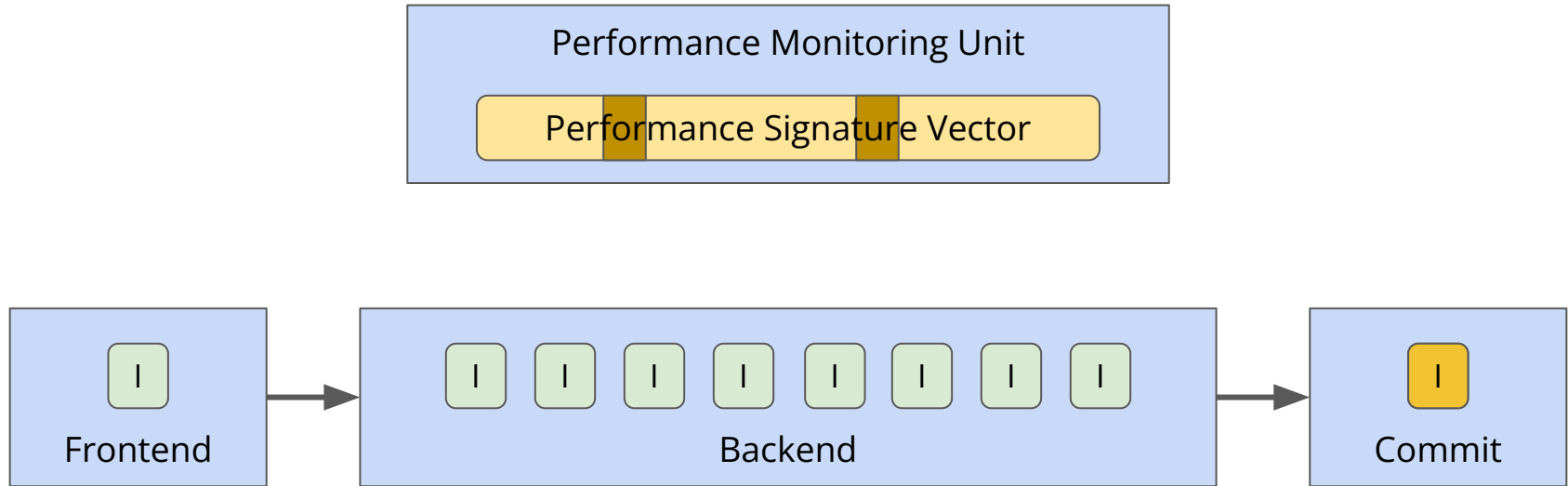


Tagged

¹ e.g. AMD IBS, Arm SPE or IBM RIS

State of the Art – Performance Profiling

Frontend Instruction Tagging¹



Instruction Legend



Normal



Stalled

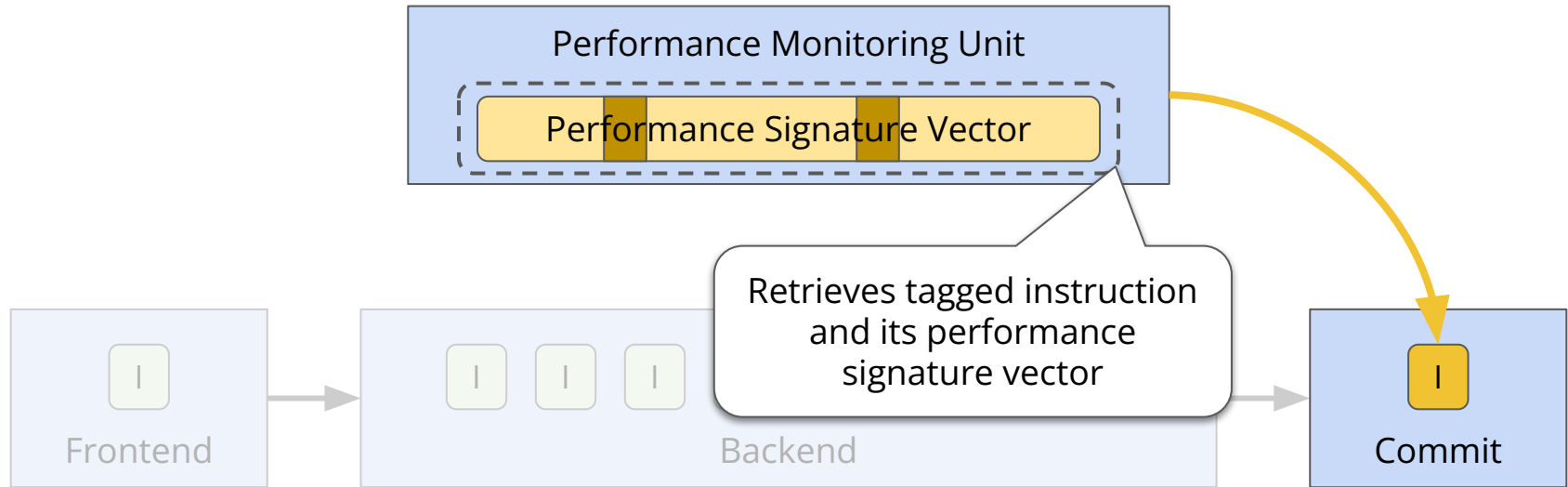


Tagged

¹ e.g. AMD IBS, Arm SPE or IBM RIS

State of the Art – Performance Profiling

Frontend Instruction Tagging¹



Instruction Legend



Normal



Stalled



Tagged

¹ e.g. AMD IBS, Arm SPE or IBM RIS

State of the Art – Performance Profiling

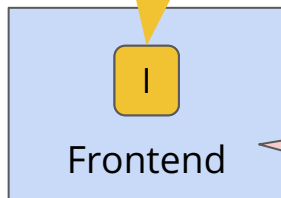
Frontend Instruction Tagging¹

Decision on which instruction to sample is made **before** it has executed

Performance Monitoring Unit

Performance Signature Vector

Tagging



Back-pressure increases probability of tagging instructions **stalling in the frontend**

Stalling instruction **responsible** for performance loss

Instruction Legend



Normal



Stalled



Tagged

¹ e.g. AMD IBS, Arm SPE or IBM RIS

State of the Art – Performance Profiling

Frontend Instruction Tagging¹

Decision which instruction to sample is made **before** it has executed

Frontend Instruction Tagging is inaccurate because it is not time-proportional

Tagged

Frontend

probability of tagging instructions **stalling in the frontend**

Stalling instruction **responsible** for performance loss

Instruction Legend



Normal



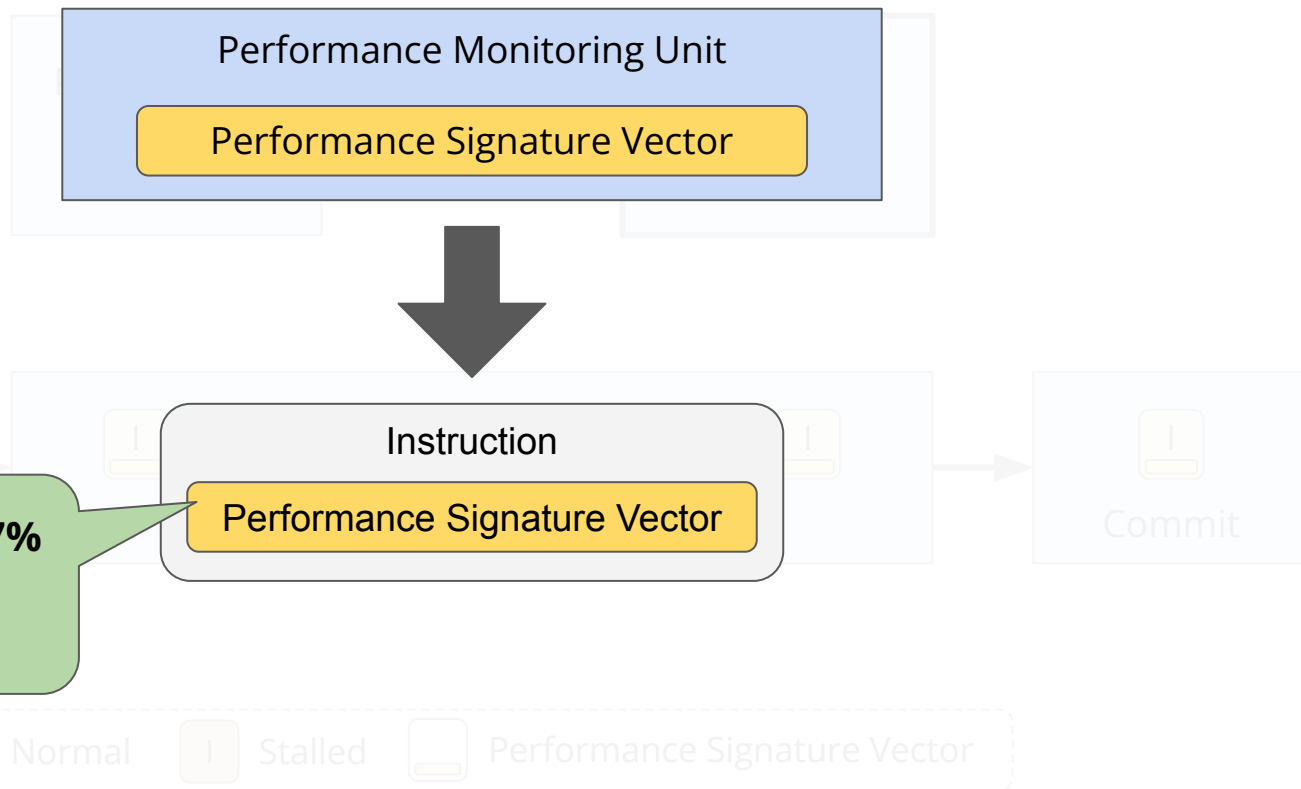
Stalled



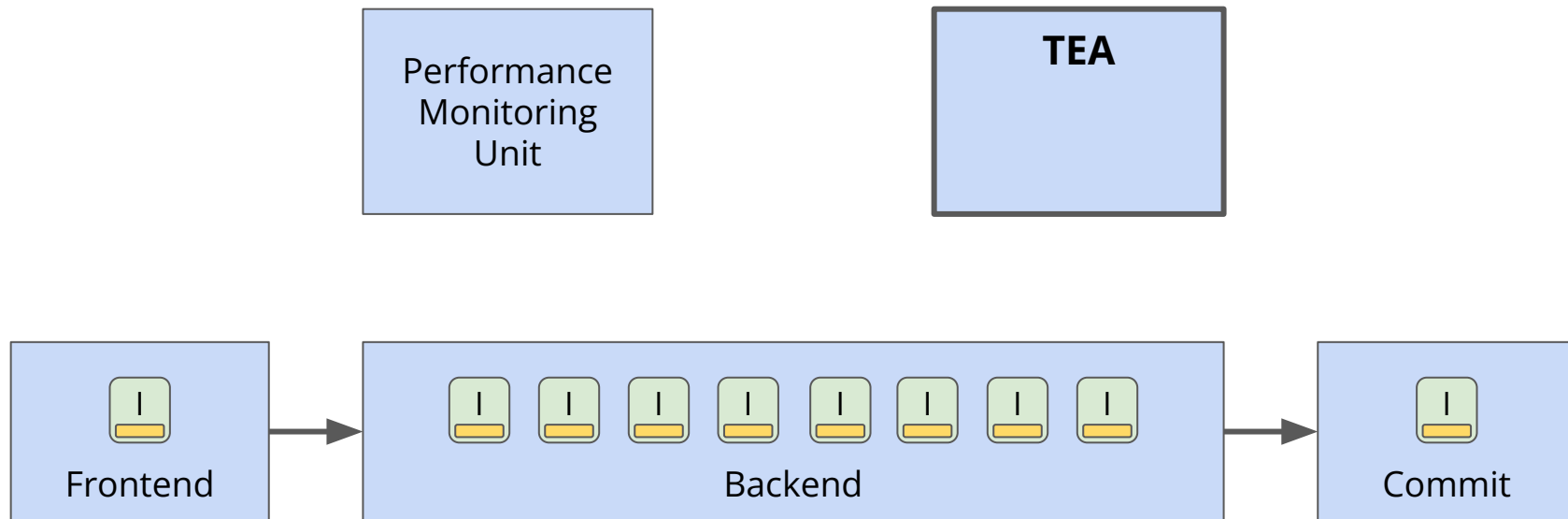
Tagged

¹ e.g. AMD IBS, Arm SPE or IBM RIS

TEA: Time-Proportional Event Analysis



TEA: Time-Proportional Event Analysis



Instruction Legend



Normal

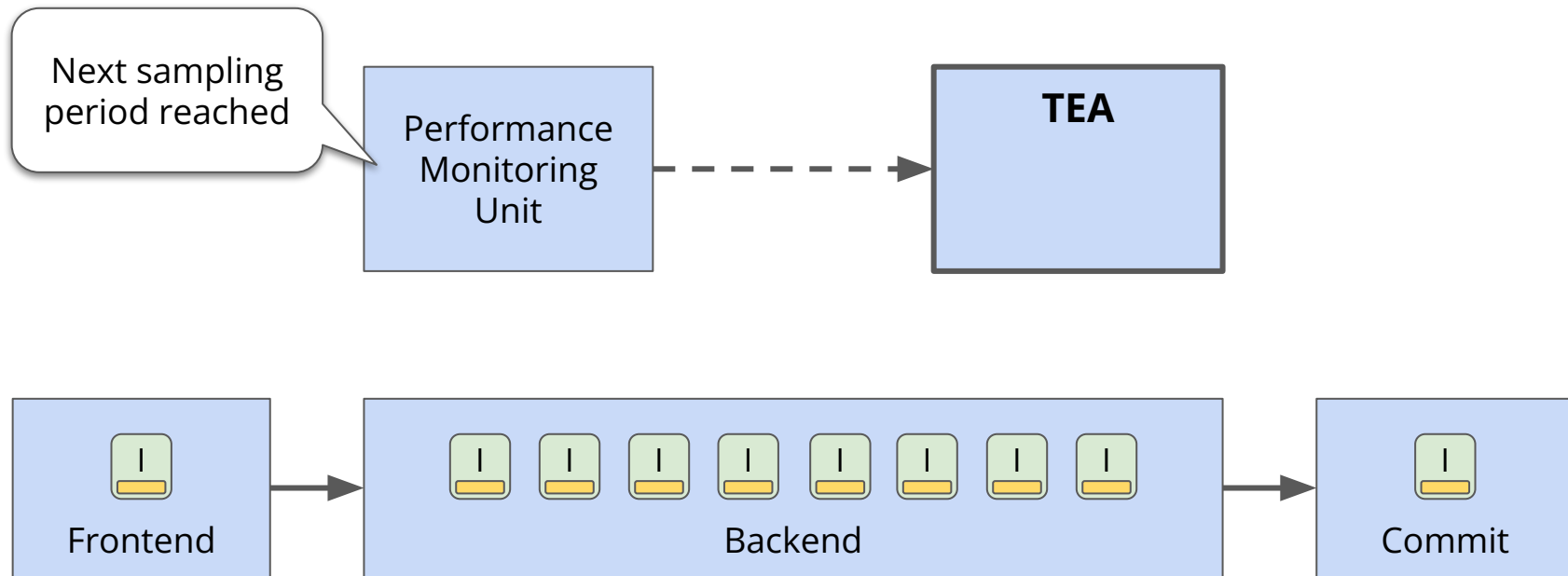


Stalled



Performance Signature Vector

TEA: Time-Proportional Event Analysis



Instruction Legend



Normal

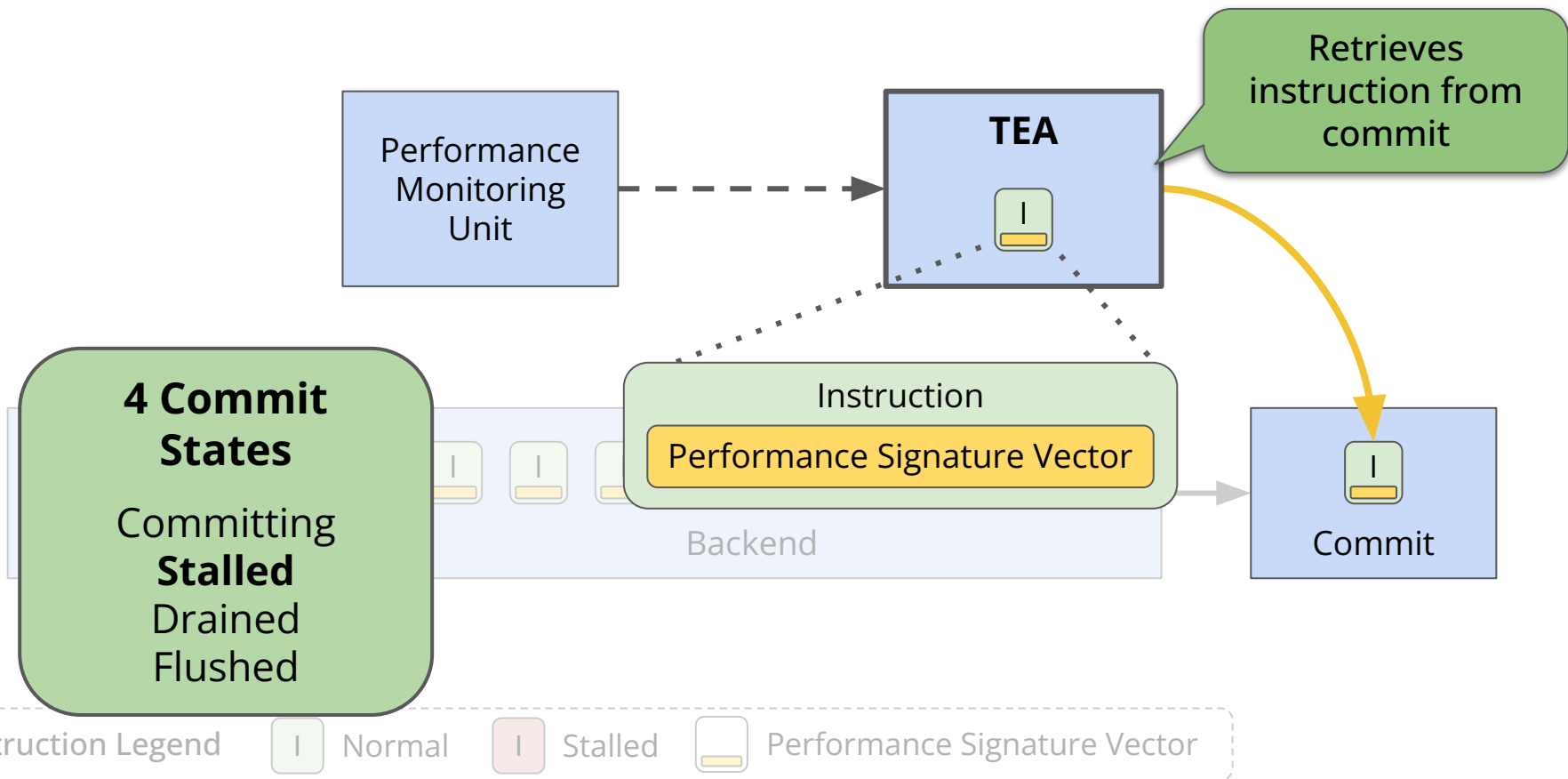


Stalled

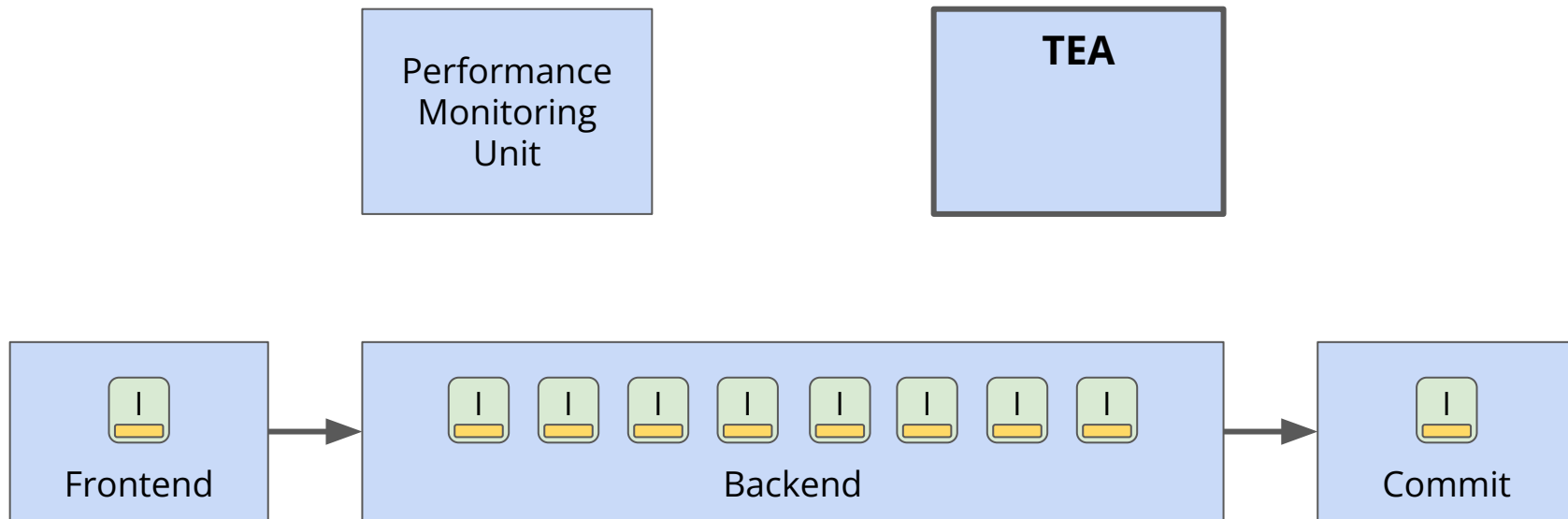


Performance Signature Vector

TEA: Time-Proportional Event Analysis



TEA: Time-Proportional Event Analysis



Instruction Legend



Normal

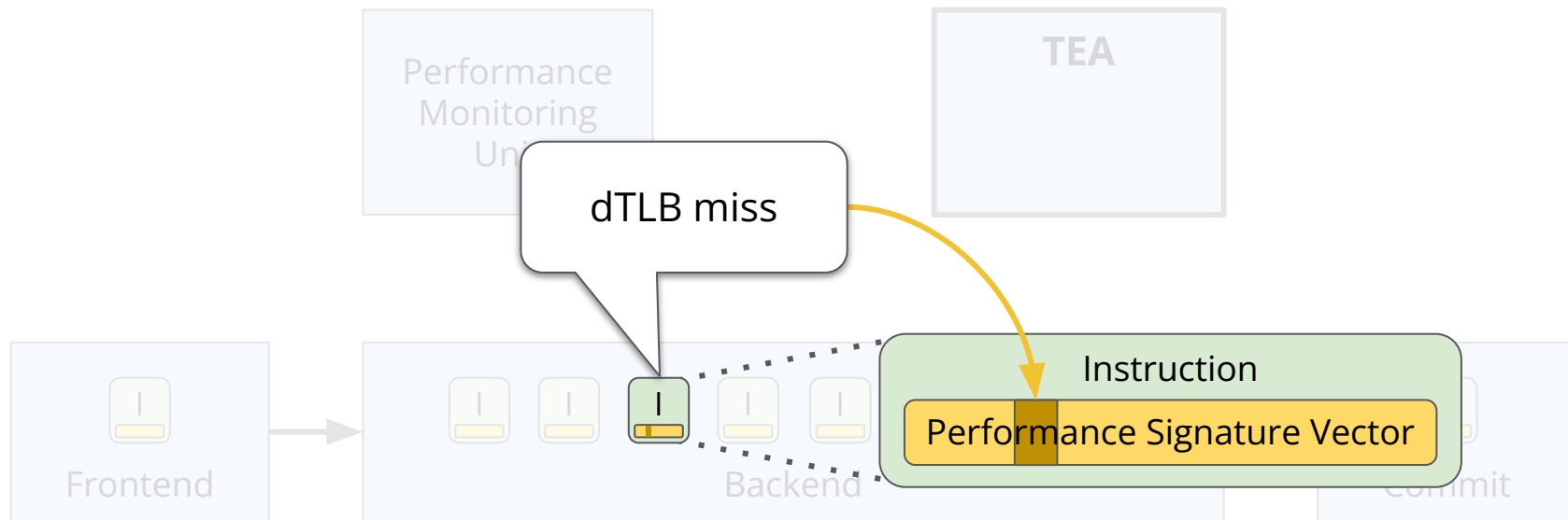


Stalled



Performance Signature Vector

TEA: Time-Proportional Event Analysis



Instruction Legend



Normal

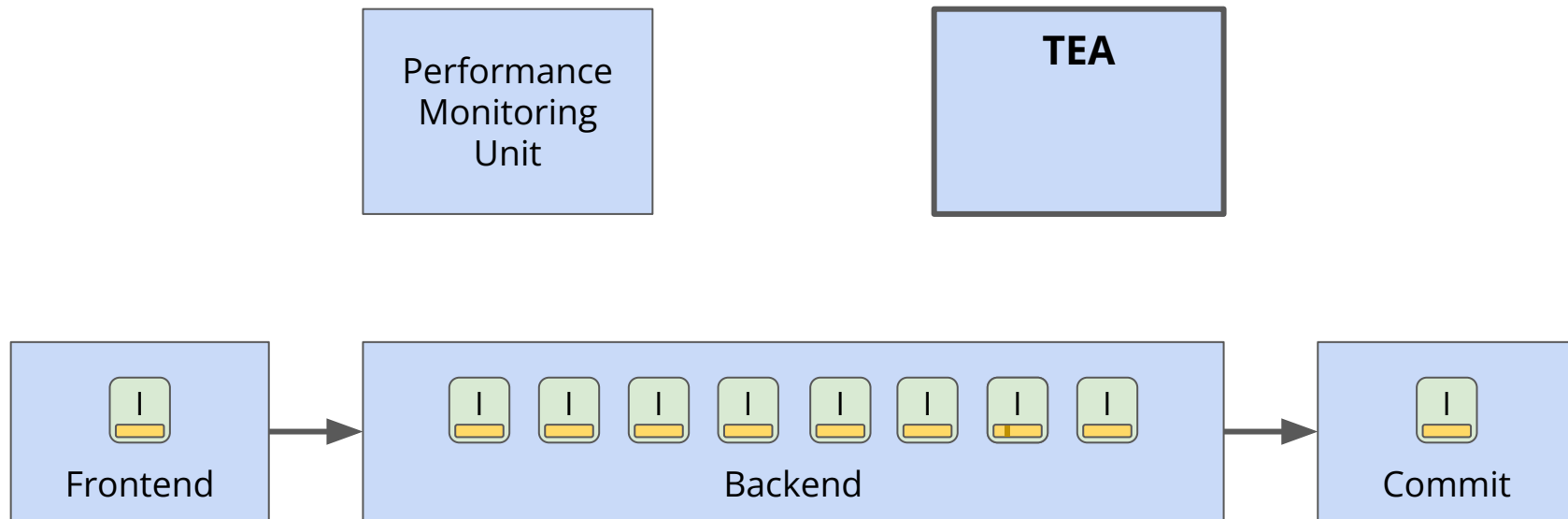


Stalled



Performance Signature Vector

TEA: Time-Proportional Event Analysis



Instruction Legend



Normal

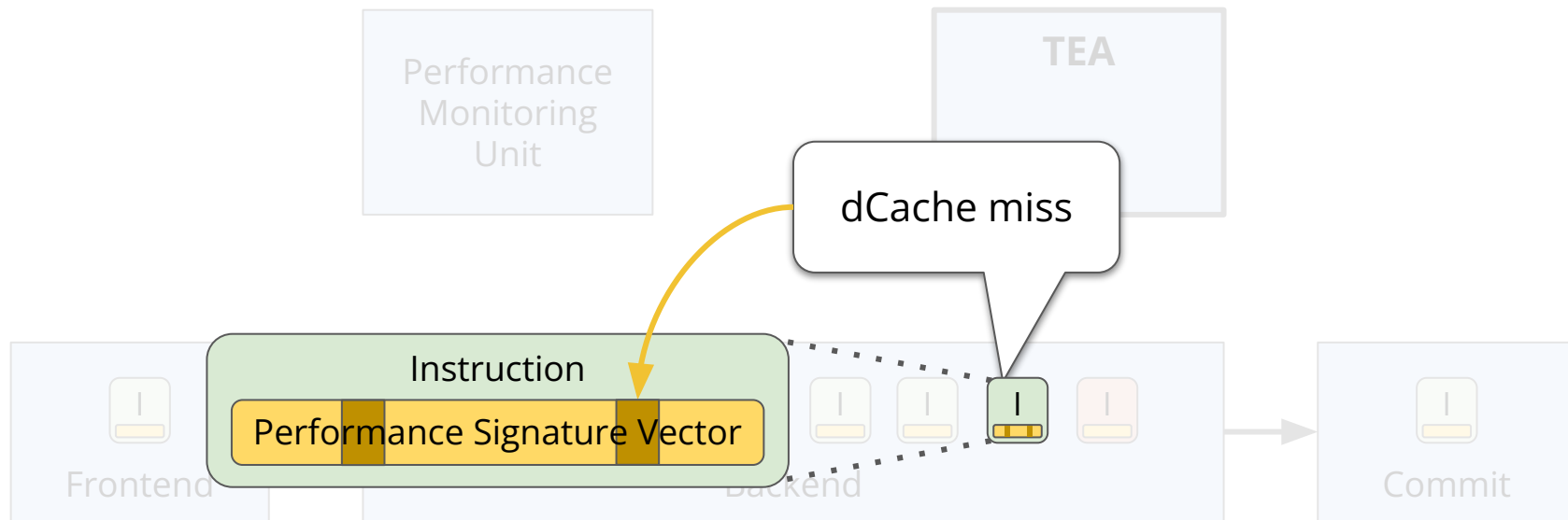


Stalled



Performance Signature Vector

TEA: Time-Proportional Event Analysis



Instruction Legend



Normal

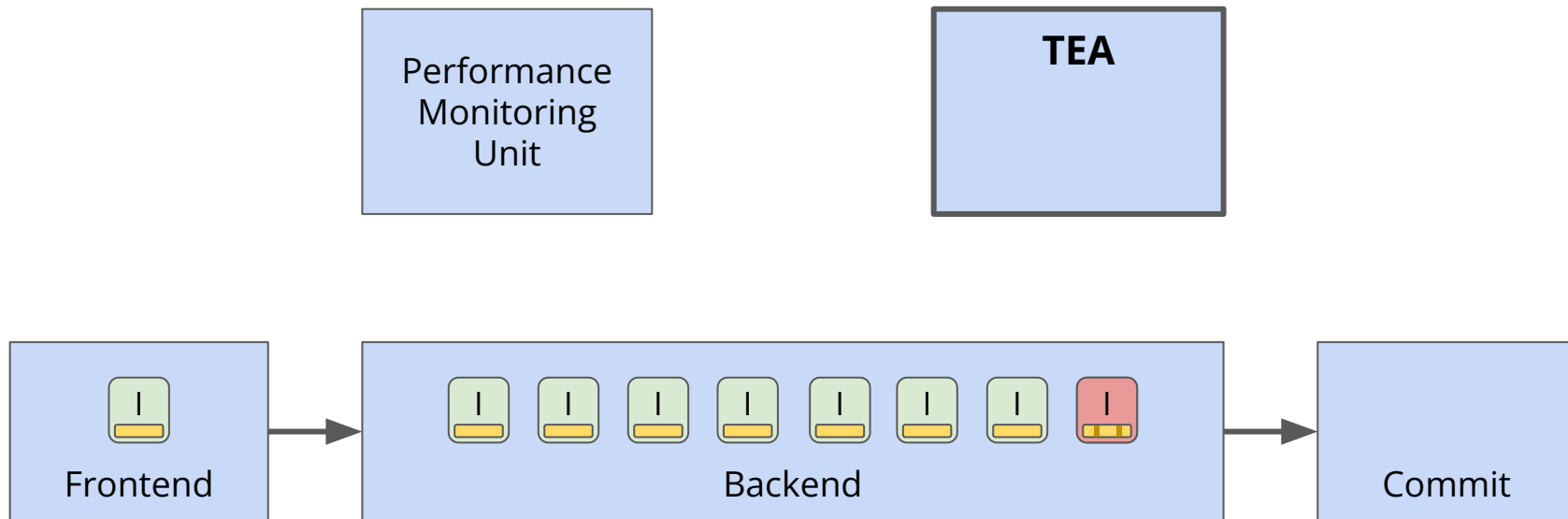


Stalled



Performance Signature Vector

TEA: Time-Proportional Event Analysis



Instruction Legend



Normal

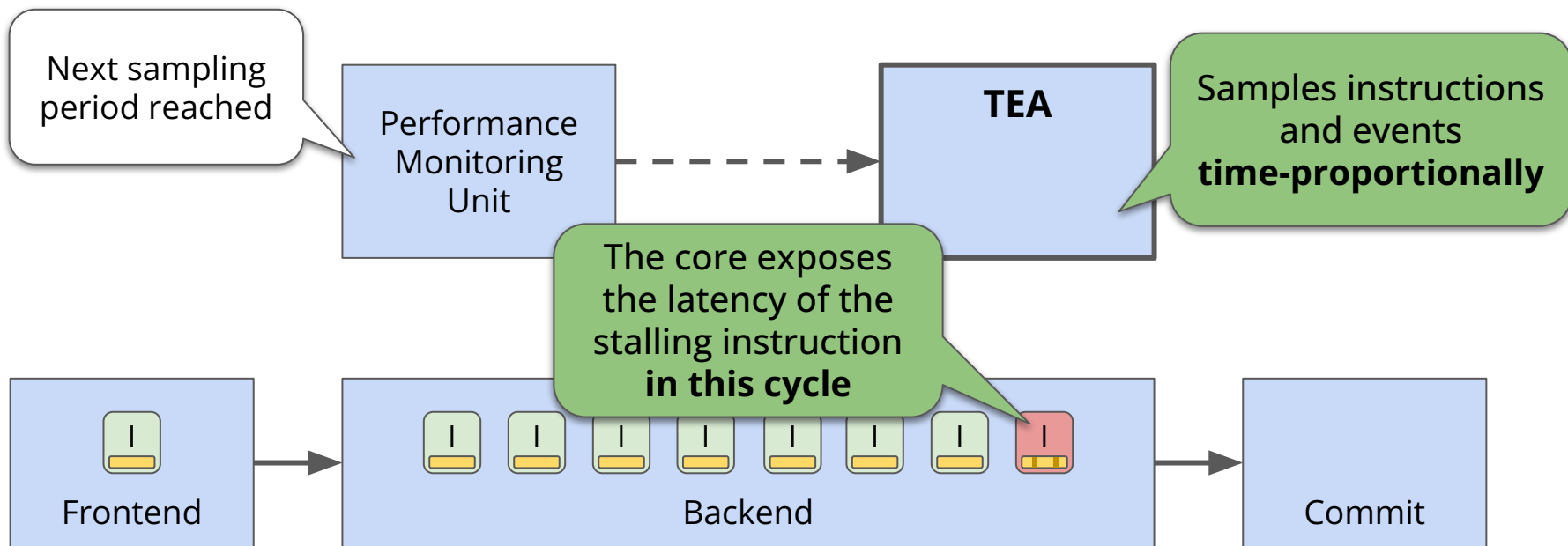


Stalled



Performance Signature Vector

TEA: Time-Proportional Event Analysis



Instruction Legend



Normal

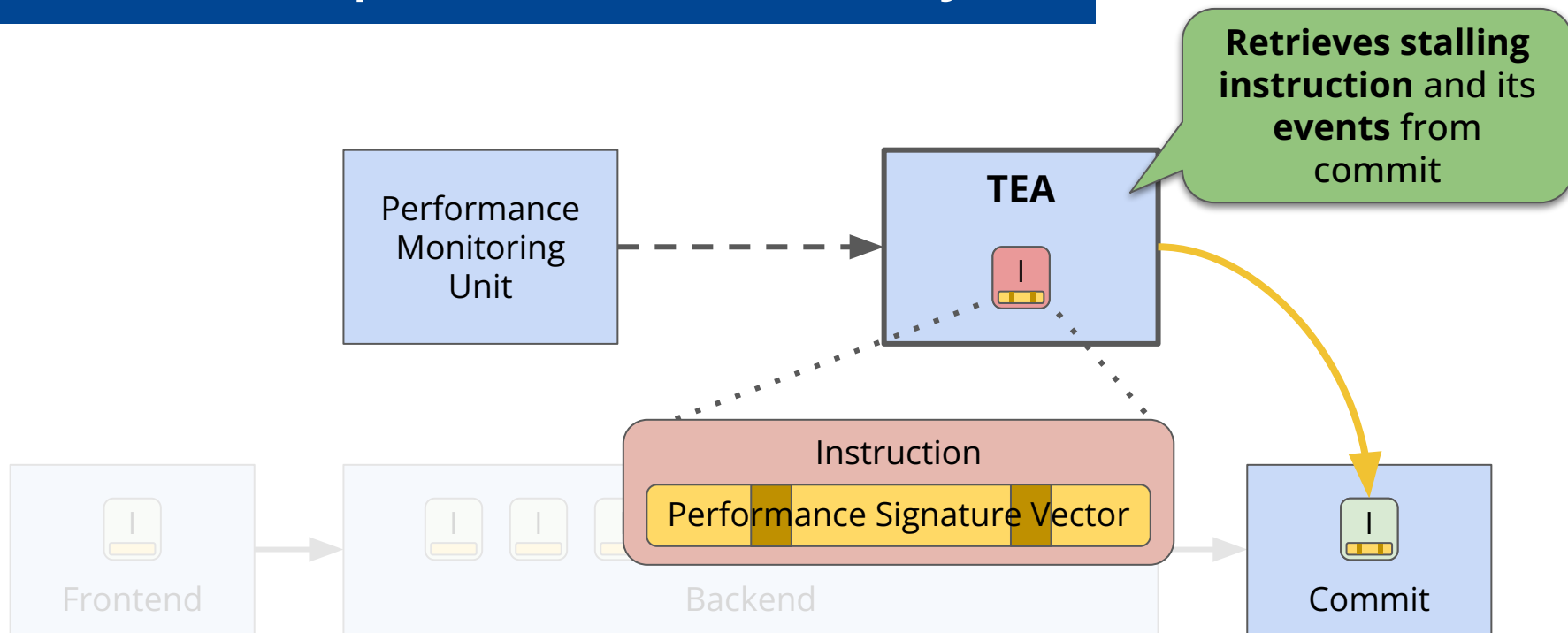


Stalled



Performance Signature Vector

TEA: Time-Proportional Event Analysis



TEA: Time-Proportional Event Analysis

**TEA time-proportionally
samples instructions and
performance events**

Retrieves stalling
instruction and its
events from
commit

Frontend

Commit

Instruction Legend



Normal



Stalled

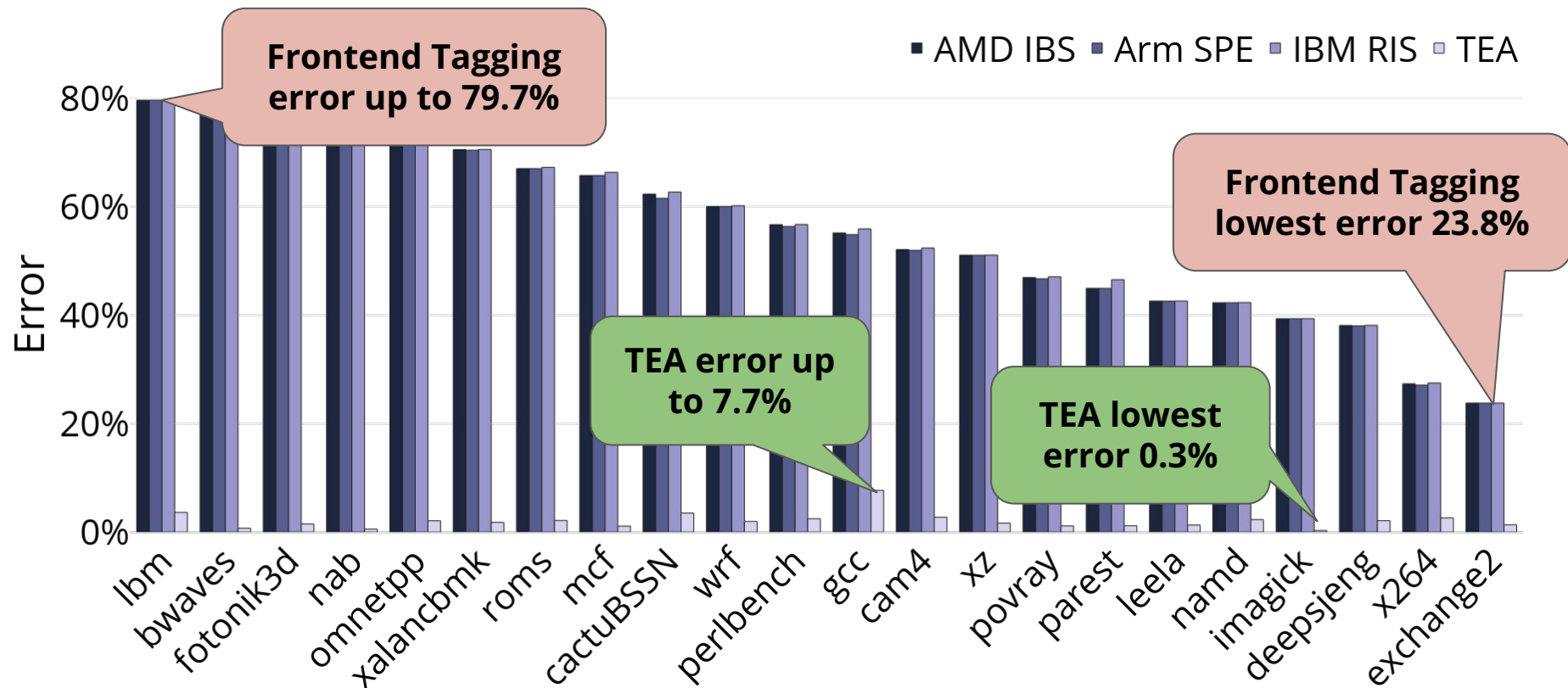


Performance Signature Vector

Experimental Setup

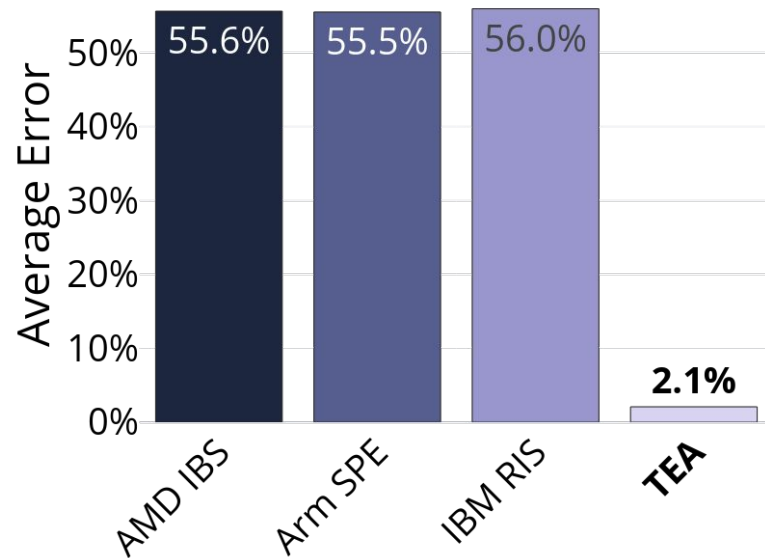
- We implement **TEA** within the **Berkeley Out-of-Order Machine** (BOOM)
- We run **SPEC CPU2017** with **reference inputs** to completion
- Our **Golden Reference** accounts for **all dynamic instructions, all performance events and all clock cycles!**

TEA is accurate



TEA is accurate

- **TEA** provides the **lowest** performance analysis **error**
- All **frontend tagging profilers** suffer from **systematic error**



Conclusion

- TEA explains which instructions your application spends time on and why
- TEA accomplishes this by time-proportionally attributing performance events to instructions
- TEA requires running the application once, is accurate, and has low storage, power, and execution time overheads

Thank You! Questions?



Björn Gottschall
bjorn.gottschall@ntnu.no

Lieven Eeckhout
lieven.eeckhout@ugent.be

Magnus Jahre
magnus.jahre@ntnu.no



Experimental Setup

We implemented in the Berkeley Out-of-Order Machine (BOOM)

- **TEA**, AMD IBS, Arm SPE and IBM RIS
- Evaluated on **FPGA using FireSim**

We created a **Golden Reference**

- Accounts for **every cycle, dynamic instruction & event**
- Analyzed **58 trillion** dynamic instructions
- Amounts to **2.7 petabytes**

We evaluated on full-system Linux

- **22 benchmarks** from **SPEC CPU2017**
- Reference inputs sets, executed to completion

Event Counts ↔ Stall Time – Pearson Correlation

